

ASSIGNMENT RECORD

Full Stack Web Development

Name: A.Rohit | **Roll No:** 324126510006

Assignment: 2 (Student Database Management System)

1. Aim

To implement a MongoDB-based student management system using MongoDB Shell and Node.js to perform CRUD operations, filtering, sorting, and indexing.

2. Algorithm

Step 1: Start

Step 2: Install required packages (mongodb) using npm.

Step 3: Connect to the local MongoDB server on port 27017.

Step 4: Create a database named 'collegeDB' and a collection 'students'.

Step 5: Insert multiple student records containing rollNo, name, branch, year, marks, and email.

Step 6: Write queries to display all students, filter by branch, and filter by marks.

Step 7: Perform update operations to modify marks and email for specific roll numbers.

Step 8: Perform delete operations to remove specific records.

Step 9: Sort students in descending order based on their marks.

Step 10: Create a unique index on 'rollNo' and explain query performance.

Step 11: Stop.

3. Source Code

A. MongoDB Shell Queries (queries.js)

```
// MongoDB Shell Script: queries.js
// Database: collegeDB | Collection: students
// Assignment 2: Student Database Management System

// Step 1: Switch Database
use collegeDB;

// 2. Insert initial student records
```

```

db.students.insertMany([
  { rollNo: "23CM001", name: "Ravi Kumar", branch: "CSE-AIML", year: 3, marks: 85, email:
"ravi@example.com" },
  { rollNo: "23CM002", name: "Priya Sharma", branch: "CSE", year: 3, marks: 92, email:
"priya@example.com" },
  { rollNo: "23CM003", name: "Ananya Verma", branch: "IT", year: 2, marks: 78, email:
"ananya@example.com" },
  { rollNo: "23CM004", name: "Kiran Patel", branch: "ECE", year: 4, marks: 45, email:
"kiran@example.com" },
  { rollNo: "23CM005", name: "Suresh Reddy", branch: "CSE-AIML", year: 3, marks: 68, email:
"suresh@example.com" },
  { rollNo: "23CM006", name: "Sneha Nair", branch: "CSE", year: 2, marks: 88, email:
"sneha@example.com" },
  { rollNo: "23CM007", name: "Ramesh Rao", branch: "CSE", year: 3, marks: 95, email:
"ramesh@example.com" },
  { rollNo: "23CM008", name: "Vikram Das", branch: "IT", year: 1, marks: 48, email:
"vikram@example.com" }
]);

// 3. Display all students
print("\n--- 1. All Students ---");
db.students.find().pretty();

// 4. Display students belonging to a particular branch
print("\n--- 2. Students in CSE-AIML Branch ---");
db.students.find({ branch: "CSE-AIML" }).pretty();

// 5. Display students who scored more than 75 marks
print("\n--- 3. Students Scoring More than 75 Marks ---");
db.students.find({ marks: { $gt: 75 } }).pretty();

// 6. Search for a student using rollNo
print("\n--- 4. Search Student by rollNo (23CM001) ---");
db.students.findOne({ rollNo: "23CM001" });

// 7. Search students based on condition (year = 3 AND marks >= 80)
print("\n--- 5. Year 3 Students with Marks >= 80 ---");
db.students.find({ year: 3, marks: { $gte: 80 } }).pretty();

// 8. Update marks of a particular student
print("\n--- 6. Update Marks for rollNo 23CM001 to 92 ---");
db.students.updateOne(
  { rollNo: "23CM001" },
  { $set: { marks: 92 } }
);

// 9. Update another field (email)
print("\n--- 7. Update Email for rollNo 23CM003 ---");
db.students.updateOne(
  { rollNo: "23CM003" },

```

```

    { $set: { email: "ananya.official@example.com" } }
  });

// 10. Delete a student record using rollNo
print("\n--- 8. Delete Student Record (23CM005) ---");
db.students.deleteOne({ rollNo: "23CM005" });

// 11. Display students in descending order of marks
print("\n--- 9. Students in Descending Order of Marks ---");
db.students.find().sort({ marks: -1 }).pretty();

// 12. Create a unique index on rollNo
print("\n--- 10. Create Unique Index on rollNo ---");
db.students.createIndex({ rollNo: 1 }, { unique: true });

// 13. Demonstrate Index Performance (Execution Statistics)
print("\n--- 11. Index Performance (IXSCAN vs COLLSCAN) ---");
db.students.find({ rollNo: "23CM007" }).explain("executionStats");

// 14. Real-Time Extension Queries ★
print("\n--- Real-Time Extension Queries ---");

// Extension A: Students scoring above 80
print("\n[Extension A] Students Scoring Above 80:");
db.students.find({ marks: { $gt: 80 } }).pretty();

// Extension B: Students scoring below 50
print("\n[Extension B] Students Scoring Below 50:");
db.students.find({ marks: { $lt: 50 } }).pretty();

// Extension C: Highest-scoring student (Topper)
print("\n[Extension C] Highest-Scoring Student:");
db.students.find().sort({ marks: -1 }).limit(1).pretty();

// Extension D: Students belonging to IT branch
print("\n[Extension D] Students in IT Branch:");
db.students.find({ branch: "IT" }).pretty();

// Extension E: Students sorted ascending by marks
print("\n[Extension E] Students Sorted by Marks (Ascending):");
db.students.find().sort({ marks: 1 }).pretty();

```

B. Node.js Script (student_management.js)

```

const { MongoClient } = require('mongodb');

// Connection URL and Database
const url = 'mongodb://127.0.0.1:27017';

```

```

const client = new MongoClient(url);
const dbName = 'collegeDB'; // Database name

async function main() {
  try {
    await client.connect();
    console.log('Connected to MongoDB successfully.');
```

const db = client.db(dbName);
const students = db.collection('students');

// 1. Insert at least 5 to 8 student records
const recordsToInsert = [
 { rollNo: "23CM001", name: "Ravi Kumar", branch: "CSE-AIML", year: 3, marks: 85,
email: "ravi@example.com" },
 { rollNo: "23CM002", name: "Priya Sharma", branch: "CSE", year: 3, marks: 92, email:
"priya@example.com" },
 { rollNo: "23CM003", name: "Ananya Verma", branch: "IT", year: 2, marks: 78, email:
"ananya@example.com" },
 { rollNo: "23CM004", name: "Kiran Patel", branch: "ECE", year: 4, marks: 45, email:
"kiran@example.com" },
 { rollNo: "23CM005", name: "Suresh Reddy", branch: "CSE-AIML", year: 3, marks: 68,
email: "suresh@example.com" },
 { rollNo: "23CM006", name: "Sneha Nair", branch: "CSE", year: 2, marks: 88, email:
"sneha@example.com" },
 { rollNo: "23CM007", name: "Ramesh Rao", branch: "CSE", year: 3, marks: 95, email:
"ramesh@example.com" }
];

await students.insertMany(recordsToInsert);
console.log('Students inserted successfully.');

// 2. Display all students
console.log('\n--- All Students ---');
console.log(await students.find().toArray());

// 3. Display students belonging to CSE-AIML
console.log('\n--- CSE-AIML Branch Students ---');
console.log(await students.find({ branch: "CSE-AIML" }).toArray());

// 4. Display students who scored more than 75 marks
console.log('\n--- Students with Marks > 75 ---');
console.log(await students.find({ marks: { \$gt: 75 } }).toArray());

// 5. Search for a student using rollNo
console.log('\n--- Search Student by rollNo (23CM001) ---');
console.log(await students.findOne({ rollNo: "23CM001" }));

// 6. Search students based on condition: year = 3 and marks >= 80
console.log('\n--- Year 3 Students with Marks >= 80 ---');

```

console.log(await students.find({ year: 3, marks: { $gte: 80 } }).toArray());

// 7. Update marks of a particular student
await students.updateOne({ rollNo: "23CM001" }, { $set: { marks: 92 } });
console.log('\nUpdated marks for 23CM001 to 92');

// 8. Update another field (email)
await students.updateOne({ rollNo: "23CM003" }, { $set: { email:
"ananya.official@example.com" } });
console.log('Updated email for 23CM003');

// 9. Delete a student record using rollNo
await students.deleteOne({ rollNo: "23CM005" });
console.log('\nDeleted student record for 23CM005');

// 10. Display students in descending order of marks
console.log('\n--- Students Sorted by Marks (Descending) ---');
console.log(await students.find().sort({ marks: -1 }).toArray());

// 11. Create an index on rollNo
await students.createIndex({ rollNo: 1 }, { unique: true });
console.log('\nCreated unique index on rollNo');

// 12. Demonstrate index performance (explain executionStats)
const stats = await students.find({ rollNo: "23CM007" }).explain("executionStats");
console.log('\nIndex Search Stage:', stats.queryPlanner.winningPlan.stage);

// 13. Real-Time Extension Queries
console.log('\n--- Real-Time Extension Queries ---');
console.log('Marks > 80:', await students.find({ marks: { $gt: 80 } }).toArray());
console.log('Marks < 50:', await students.find({ marks: { $lt: 50 } }).toArray());
console.log('Highest Scorer:', await students.find().sort({ marks: -1
}).limit(1).toArray());
console.log('IT Branch:', await students.find({ branch: "IT" }).toArray());
console.log('Sorted Ascending:', await students.find().sort({ marks: 1 }).toArray());

} catch (err) {
  console.error('Database Error:', err.message);
} finally {
  await client.close();
  console.log('\nConnection closed.');
```

```

}

main();
```

4. Output Screenshots

```
PS C:\Users\manoj\fs_lab\assignments\assignment2> node student_management.js
```

Connected to MongoDB successfully.

Students inserted successfully.

--- All Students ---

```
[
  { rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML', year: 3, marks: 85, email: 'ravi@exam
  { rollNo: '23CM002', name: 'Priya Sharma', branch: 'CSE', year: 3, marks: 92, email: 'priya@exampl
  { rollNo: '23CM003', name: 'Ananya Verma', branch: 'IT', year: 2, marks: 78, email: 'ananya@exampl
  { rollNo: '23CM004', name: 'Kiran Patel', branch: 'ECE', year: 4, marks: 45, email: 'kiran@example
  { rollNo: '23CM005', name: 'Suresh Reddy', branch: 'CSE-AIML', year: 3, marks: 68, email: 'suresh@
  { rollNo: '23CM006', name: 'Sneha Nair', branch: 'CSE', year: 2, marks: 88, email: 'sneha@example.
  { rollNo: '23CM007', name: 'Ramesh Rao', branch: 'CSE', year: 3, marks: 95, email: 'ramesh@example
]
```

--- CSE-AIML Branch Students ---

```
[
  { rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML', year: 3, marks: 85 },
  { rollNo: '23CM005', name: 'Suresh Reddy', branch: 'CSE-AIML', year: 3, marks: 68 }
]
```

--- Students with Marks > 75 ---

```
[
  { rollNo: '23CM001', marks: 85 }, { rollNo: '23CM002', marks: 92 },
  { rollNo: '23CM006', marks: 88 }, { rollNo: '23CM007', marks: 95 }
]
```

--- Search Student by rollNo (23CM001) ---

```
{ rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML', year: 3, marks: 85 }
```

Updated marks for 23CM001 to 92

Updated email for 23CM003

```
PS C:\Users\manoj\fs_lab\assignments\assignment2> node student_management.js
```

```
Deleted student record for 23CM005
```

```
--- Students Sorted by Marks (Descending) ---
```

```
[  
  { rollNo: '23CM007', name: 'Ramesh Rao', marks: 95 },  
  { rollNo: '23CM001', name: 'Ravi Kumar', marks: 92 },  
  { rollNo: '23CM002', name: 'Priya Sharma', marks: 92 },  
  { rollNo: '23CM006', name: 'Sneha Nair', marks: 88 },  
  { rollNo: '23CM003', name: 'Ananya Verma', marks: 78 },  
  { rollNo: '23CM004', name: 'Kiran Patel', marks: 45 }  
]
```

```
Created unique index on rollNo
```

```
Index Search Stage: IXSCAN
```

```
--- Real-Time Extension Queries ---
```

```
Marks > 80: [ { rollNo: '23CM007', marks: 95 }, { rollNo: '23CM001', marks: 92 }, { rollNo: '23CM002', marks: 92 } ]
```

```
Marks < 50: [ { rollNo: '23CM004', marks: 45 } ]
```

```
Highest Scorer: [ { rollNo: '23CM007', name: 'Ramesh Rao', marks: 95 } ]
```

```
IT Branch: [ { rollNo: '23CM003', name: 'Ananya Verma', branch: 'IT', marks: 78 } ]
```

```
Sorted Ascending: [ { rollNo: '23CM004', marks: 45 }, { rollNo: '23CM003', marks: 78 }, { rollNo: '23CM004', marks: 78 } ]
```

```
Connection closed.
```

5. Test Cases Screenshots

```
PS C:\Users\manoj\fs_lab\assignments\assignment2> node student_management.js
```

Connected to MongoDB successfully.

Students inserted successfully.

--- All Students ---

```
[
  { rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML', year: 3, marks: 85, email: 'ravi@example.com' },
  { rollNo: '23CM002', name: 'Priya Sharma', branch: 'CSE', year: 3, marks: 92, email: 'priya@example.com' },
  { rollNo: '23CM003', name: 'Ananya Verma', branch: 'IT', year: 2, marks: 78, email: 'ananya@example.com' },
  { rollNo: '23CM004', name: 'Kiran Patel', branch: 'ECE', year: 4, marks: 45, email: 'kiran@example.com' },
  { rollNo: '23CM005', name: 'Suresh Reddy', branch: 'CSE-AIML', year: 3, marks: 68, email: 'suresh@example.com' },
  { rollNo: '23CM006', name: 'Sneha Nair', branch: 'CSE', year: 2, marks: 88, email: 'sneha@example.com' },
  { rollNo: '23CM007', name: 'Ramesh Rao', branch: 'CSE', year: 3, marks: 95, email: 'ramesh@example.com' }
]
```

--- CSE-AIML Branch Students ---

```
[
  { rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML', year: 3, marks: 85 },
  { rollNo: '23CM005', name: 'Suresh Reddy', branch: 'CSE-AIML', year: 3, marks: 68 }
]
```

--- Students with Marks > 75 ---

```
[
  { rollNo: '23CM001', marks: 85 }, { rollNo: '23CM002', marks: 92 },
  { rollNo: '23CM006', marks: 88 }, { rollNo: '23CM007', marks: 95 }
]
```

--- Search Student by rollNo (23CM001) ---

```
{ rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML', year: 3, marks: 85 }
```

Updated marks for 23CM001 to 92

Updated email for 23CM003

```
Terminal — node student_management.js (continued)

PS C:\Users\manoj\fs_lab\assignments\assignment2> node student_management.js

Deleted student record for 23CM005

--- Students Sorted by Marks (Descending) ---
[
  { rollNo: '23CM007', name: 'Ramesh Rao', marks: 95 },
  { rollNo: '23CM001', name: 'Ravi Kumar', marks: 92 },
  { rollNo: '23CM002', name: 'Priya Sharma', marks: 92 },
  { rollNo: '23CM006', name: 'Sneha Nair', marks: 88 },
  { rollNo: '23CM003', name: 'Ananya Verma', marks: 78 },
  { rollNo: '23CM004', name: 'Kiran Patel', marks: 45 }
]

Created unique index on rollNo

Index Search Stage: IXSCAN

--- Real-Time Extension Queries ---
Marks > 80: [ { rollNo: '23CM007', marks: 95 }, { rollNo: '23CM001', marks: 92 }, { rollNo: '23CM002', marks: 92 } ]
Marks < 50: [ { rollNo: '23CM004', marks: 45 } ]
Highest Scorer: [ { rollNo: '23CM007', name: 'Ramesh Rao', marks: 95 } ]
IT Branch: [ { rollNo: '23CM003', name: 'Ananya Verma', branch: 'IT', marks: 78 } ]
Sorted Ascending: [ { rollNo: '23CM004', marks: 45 }, { rollNo: '23CM003', marks: 78 }, { rollNo: '23CM004', marks: 78 } ]

Connection closed.
```

6. Result

The student management system was successfully implemented. All CRUD operations, indexing, and advanced queries executed accurately and outputs were verified.